

ISIMA 3^{ème} année – MODL/C++

TP 1 (2 séances) : Rappels

Préambule

Ce TP vous fera (re)découvrir deux outils de développement logiciel : **CMake** et les **tests unitaires**.

CMake (Cross-platform Make) est un outil permettant de compiler vos codes sur diverses plateformes. Son rôle est de générer le *makefile* approprié en tenant compte de la configuration particulière de la machine sur laquelle il est exécuté (chemin vers les bibliothèques, options de compilation pour une architecture donnée...). Il fonctionne avec un fichier de configuration qui vous est donné pour ce TP : **CMakeLists.txt**. Nous vous encourageons fortement à le consulter pour comprendre son fonctionnement (la documentation de référence de CMake se trouve à l'adresse suivante : <https://www.cmake.org>).

Pour ce TP, vous n'avez plus qu'à générer le *makefile* de votre projet. Utilisez les commandes suivantes depuis le répertoire où se trouve le fichier **CMakeLists.txt** :

```
cd build
cmake ..
```

À présent vous pouvez taper **make** dans le répertoire **build** pour reconstruire le projet après chaque modification du code source. Vos fichiers sources doivent être placés dans le répertoire **src**. **N'oubliez pas de les déclarer dans le fichier de configuration CMakeLists.txt.**

Lors de sa première exécution, **make** est relativement lent. En effet, il compile la **bibliothèque de tests unitaires Catch** (pour plus d'informations : <https://github.com/catchorg/Catch2>). Les tests unitaires sont utilisés pour vérifier qu'un élément logiciel (fonction, méthode, objet...) produit les résultats que l'on attend de lui. On parle de *Test-Driven Development* dès lors que les tests unitaires sont écrits en amont du code source et guident le développement de l'application. C'est au développeur d'identifier les cas où l'application peut être mise en échec.

Dans notre cas, des tests élémentaires vérifiant le bon fonctionnement des questions de l'exercice 1 vous sont fournis. Vous les trouverez dans le répertoire **test** du projet. La syntaxe des tests est extrêmement simple, il suffit de donner un nom au test et à la batterie qui le contient, avant de faire des assertions pour vérifier le résultat d'un appel de méthode par exemple. **Il faut bien sûr que l'interface du code que vous développez respecte celle utilisée dans les tests.** Par exemple, le code suivant :

```
TEST_CASE("TP1_Polaire::Constructeur") {
    const double mod = 24.0;
    const double arg = 12.0;

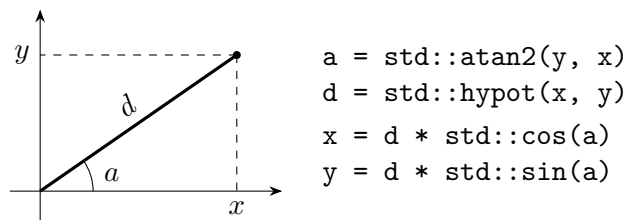
    Polaire p(mod, arg);

    REQUIRE(p.getArg() == Approx(arg));
    REQUIRE(p.getMod() == Approx(mod));
}
```

vérifie que le constructeur de la classe **Polaire** remplit correctement les attributs des instances qu'il crée et que les accesseurs **getArg()** et **getMod()** renvoient les valeurs attendues.

Exercice 1 – Complexe

- 1) Définir une classe `Complexe` proposant une méthode abstraite `afficher()`. Dériver deux classes, `Algebrique` et `Polaire`, représentant un nombre sous forme respectivement algébrique et polaire (angle en radians). Offrir un jeu de fonctionnalités minimal à ces classes. Tests 1–11
- 2) Surcharger l'opérateur de flux de sortie de manière à ce qu'il exploite l'héritage défini précédemment (cf. méthode `afficher()`). Test 12
- 3) Proposer des passerelles de conversion entre les deux classes concrètes. Deux approches sont possibles, il vous est demandé de les implémenter :
 - (i) Proposer un constructeur dans chacune des deux classes concrètes qui permette la conversion à partir de l'autre classe. Un problème d'interdépendance entre classes vous obligera à faire des déclarations anticipées (*forward declarations*). Tests 13–14
 - (ii) Proposer une méthode virtuelle dans la classe `Complexe` et ses filles pour convertir un nombre dans le type `Algebrique`. Tests 15–17



https://fr.wikipedia.org/wiki/Coordonn%C3%A9es_polaires

- 4) Dans la *sandbox* (`sandbox/main.cpp`), stocker des instances de `Algebrique` et de `Polaire` dans un vecteur STL et afficher son contenu. Essayer :
 - `std::vector<Complexe>`;
 - `std::vector<Complexe*>`.
- 5) Au vu de la question précédente, définir une classe `Ensemble` utilisant la classe `std::vector` et contenant un ensemble de nombres (qui peuvent être de types différents). Cette classe devra fournir des itérateurs, à la manière STL (méthodes `begin()` et `end()`, type `iterator`), et une méthode `size()` qui renvoie le nombre d'éléments stockés. Réutiliser au maximum les méthodes déjà existantes de la classe `std::vector`. Tests 18–19
- 6) Proposer une fonction qui calcule la moyenne d'un ensemble de nombres. Tests 20–21

Proposer ensuite deux versions foncteurs de cette fonction qui renvoient la moyenne sous la forme respectivement d'un `Algebrique` et d'un `Polaire`. Tests 22–23

Exercice 2 – Mandelbrot

Le but de cet exercice est de produire une image qui représente l'ensemble de Mandelbrot (https://fr.wikipedia.org/wiki/Ensemble_de_Mandelbrot).

Le code fourni dans `ex2_mandelbrot/src/main.cpp` permet l'écriture d'un fichier BMP en utilisant la bibliothèque CImg (<https://cimg.eu>). La fonction `mandelbrot()` doit modifier l'image afin de créer une représentation de l'ensemble de Mandelbrot dans une région donnée.

Elle accepte 4 paramètres :

- `img` : l'image dans laquelle écrire ;
- `maxIter` : le nombre maximum d'itérations par pixel ;
- `min` : les coordonnées (partie réelle/imaginaire) minimales ;
- `max` : les coordonnées (partie réelle/imaginaire) maximales.

1. Compléter la fonction `mandelbrot()` pour calculer la couleur de chaque pixel (`colour`) selon la vitesse de divergence de la suite.

L'algorithme : pour un pixel donné (complexe c), calculer le complexe $z_{i+1} = z_i^2 + c$ pour i de 0 à `maxIter`, avec $z_0 = 0$, tant que z_i ne diverge pas (son module ne doit pas dépasser le seuil de divergence fixé à 2). On peut alors affecter comme couleur au pixel le rapport entre le nombre d'itérations effectuées et `maxIter`.